

CPU 微架构性能指标五场景横向对比

测试环境

- **CPU:** AMD EPYC 9K65 192-Core Processor (Zen 4 / Genoa)
- **拓扑:** 1 Socket, 16 Core, 32 Thread (SMT)
- **缓存:** L1d 48KB/core, L2 1MB/core, L3 32MB shared
- **虚拟化:** KVM
- **内核:** 6.8.0-117-generic
- **cpu频率策略:** KVM 云虚拟机不开放 CPU 调频驱动
- **备注:** LLC-load-misses 在此 KVM 环境中不支持，缓存 Miss Rate 仅基于 cache-misses/cache-references

原始数据汇总

指标	int64	matrixprod	read64	rand-set	queens
cycles	93,665,965,362	93,103,195,894	93,722,101,233	88,972,359,917	93,519,255,644
instructions	302,078,844,028	91,759,543,609	339,701,916,425	417,670,128,035	178,071,771,580
cache-references	51,397,767	22,508,807,534	54,567,880,502	2,154,403,145	49,141,631
cache-misses	1,855,415	5,766,074	2,984,613	45,058,758	1,789,891
L1-dcache-load-misses	10,831,054	11,455,794,386	740,230,932	1,169,333,685	11,074,512
L1-icache-load-misses	10,993,785	9,661,916	10,751,623	94,008,665	10,480,161
LLC-load-misses	N/S	N/S	N/S	N/S	N/S
branch-instructions	3,659,290,723	11,170,710,523	24,403,321,615	58,918,212,866	18,802,047,090
branch-misses	7,916,990	92,752,144	4,815,026	42,869,308	2,159,717,509

dTLB-load-misses	58,911	91,458	503,148,504	28,617,334	53,114
context-switches	61	53	66	248	64
cpu-migrations	1	0	0	1	0

衍生指标对比

衍生指标	计算公式	int64	matrixprod	read64	rand-set	queens
IPC	instructions / cycles	3.23	0.99	3.62	4.69	1.90
L1 DCache Miss Rate	L1-dcache-load-misses / cache-references	21.1%	50.9%	1.36%	54.3%	22.5%
LLC Miss Rate	LLC-load-misses / cache-references	N/S	N/S	N/S	N/S	N/S
Cache Miss Rate	cache-misses / cache-references	3.61%	0.03%	0.01%	2.09%	3.64%
分支预测失败率	branch-misses / branch-instructions	0.22%	0.83%	0.02%	0.07%	11.49%
TLB Miss Rate	dTLB-load-misses / cache-references	0.11%	0.00%	0.92%	1.33%	0.11%

差异分析

1. 整数运算 (int64) — ALU 流水线效率考察

- 分析：**int64 负载是紧凑循环中的 64 位整数运算。IPC 达到 3.23，受限於 stress-ng 循环开销，尚未触及 Zen 4 的 ALU 吞吐上限，后端执行单元（ALU 端口）是瓶颈。前端取指/解码毫无压力，因为icache miss和分支失败率都很低，数据也完全可以被 L1 Cache 覆盖。CPU 的限制因素纯粹是每个周期能发射多少条整数微操作，即执行端口的物理上限。
- 结论：**纯整数运算是典型的"计算绑定"型负载，瓶颈在后端 ALU 吞吐

2. 矩阵乘法 (matrixprod) — 浮点单元与 L1 Cache 考察

- 分析：**matrixprod 是本组数据中 IPC 最低的场景，根本原因是 L1 数据缓存严重受限。矩阵乘法内层循环按行遍历一个矩阵、按列遍历另一个矩阵，列向访问的 stride 等于行宽，远超 64B 的 Cache Line 大小。每次列向读取都可能导致 L1 Cache Line 被替换出去，50.9% 的 L1 Miss Rate 意味着大量周期浪费在等待 L2/L3 返回数据上。后端 ALU 因等待操作数而间歇停顿，导致 IPC 降至 1 以下。

- 对比 int64：同为计算密集型，int64 数据集小可全部放进 L1（Miss 21%），而 matrixprod 的矩阵太大无法放进 L1（Miss 51%），这是两者 IPC 差异的核心原因。
- **结论：**矩阵乘法是"L1 访存绑定"型负载，瓶颈在缓存行替换与数据预取效率

3. 连续访存 (read64) — Cache 层级与内存带宽考察

- **分析：**read64 以顺序方式遍历 1GB 内存。看似应该有大量 Cache Miss，但 L1 DCache Miss Rate 仅 1.36%，这是因为 CPU 的硬件预取器能识别顺序访问模式，提前将下一批 Cache Line 从内存拉入 L1/L2。预取器的完美预测使数据几乎总在 L1 中准备好，后端保持高吞吐（IPC 3.62）。但 1GB 的内存足迹带来了 TLB 压力。标准 4KB 页表下，1GB 需要 256K 个页表项，远超 L1 DTLB 和 L2 STLB 的容量。不过 TLB Miss 的代价比 Cache Miss 小得多，所以对 IPC 的影响有限。瓶颈在内存带宽：顺序读取 1GB 持续占用 DRAM 带宽，但带宽上限并没有反映在当前的 perf 事件中。连续访存是"带宽受限"型负载（而非延迟受限），硬件预取掩盖了延迟，使 IPC 保持高位，但持续吞吐量最终受 DRAM 带宽上限约束。
- **结论：**连续访存是"内存带宽绑定"型负载，硬件预取器效果好，但 TLB 压力较大

4. 随机访存 (rand-set) — TLB 与 Cache Miss 考察

- **分析：**sys 时间 7.85s 其他四个场景的 sys 时间均 < 0.12s，随机写入 512MB 内存时，进程访问的虚拟地址跨越大量 4KB 页面。当首次访问某个页面时，CPU 触发 Page Fault，内核需要分配一个物理页帧，建立虚拟地址到物理地址的页表映射，刷新 TLB 这些操作全部在内核态完成，perf 统计到的 instructions 和 cycles 包含了大量内核态的指令执行。内核代码路径简单（线性页表操作），所以 IPC 看似很高，但 CPU 实际上在做非有效计算。rand-set 没有 --vm-keep，stress-ng 每轮会释放再重分配 512MB 页面，持续触发 Page Fault，IPC 4.69 并非说明随机访存效率高，而是内核缺页处理指令占据了大部分统计，掩盖了用户态低效的随机访存。
- 与 read64 对比：read64 只读不写且使用 --vm-keep（复用已分配页面），没有 Page Fault 问题；rand-set 是随机写入新页面，不断触发缺页中断。
- **结论：**随机访存是"内存延迟绑定 TLB 绑定"型负载，硬件预取器完全失效，瓶颈在访存延迟而非带宽

5. 分支预测(queens) — 分支预测器能力考察

- **分析：**queens 使用回溯算法求解 N 皇后问题，包含大量 if-else 和递归调用。分支预测失败率高达 11.49%（其他场景均 < 1%），是五场景中唯一的前端瓶颈场景。分支预测失败的代价：大量前端工作量被浪费 访存不是问题，执行单元也没有压力。唯一的瓶颈就是分支预测器无法有效预测回溯算法的不规则分支模式。
- **结论：**N-queens 是"分支预测绑定"型负载，回溯算法产生大量不可预测的条件跳转，分支预测器成为最大短板